

Department of Computer Science and Engineering

Indian Institute of Technology Guwahati

Lab 0

First Boot: Meet Luit

CS3106L - Operating Systems: Concepts and Construction

BrahmaputraOS / kernel *Luit* - Student Base v1.0

Completion checkpoint | Suggested time: 2.5–3 hours

No kernel feature is implemented in this lab.

Dr. Satyajit Das

Department of Computer Science and Engineering, IIT Guwahati

Contents

1	Purpose of Lab 0	3
1.1	Learning outcomes	3
1.2	What Lab 0 is not	3
1.3	Suggested working plan	3
2	The System Before You Touch It	4
2.1	Two machines and four important boundaries	4
2.2	The three artifacts you should always recognise	4
3	Phase A - Verify the Development Tools	5
4	Phase B - Unpack, Inspect, and Build	5
4.1	Unpack the student release	5
4.2	Inspect before building	5
4.3	Perform a clean build	6
4.4	Classify the generated artifacts	6
5	Phase C - Boot Luit	6
5.1	Start the virtual machine	6
5.2	Boot path: Linux and Luit	7
6	Phase D - Explore the Unix Interface	7
6.1	Create data before trying file-processing commands	7
6.2	Compose programs	8
6.3	Understand one pipe	8
7	Phase E - Observe Processes and Memory	9
7.1	Process states	9
7.2	Virtual and physical memory	9
8	Phase F - Test Persistence Carefully	9
8.1	Create a file and stop after the write is complete	10
8.2	Understand what destroys the disk image	10
9	Phase G - Learn the Source Organisation	10
9.1	Top-level map	11
9.2	Subsystem map inside the source	11
9.3	Use a three-pass reading method	12
9.4	Guided tour 1: read the boot map	12
9.5	Guided tour 2: locate a process	12

9.6 Guided tour 3: trace one output operation	12
10 Phase H - Complete One Edit–Build–Boot–Run Cycle	13
10.1 Create <code>user/hello.c</code>	13
10.2 Add the program to the disk-image build	13
10.3 Build and observe what changed	14
10.4 Boot and run	14
11 Host and Guest Quick Reference	15
12 What to Submit	15
13 Getting Unstuck	15
14 The Main Ideas to Carry Forward	16

1 Purpose of Lab 0

This is the zeroth laboratory. You will not add a kernel feature today. Instead, you will learn how the complete Luit system is organised and practise the development cycle that you will use throughout the course:

inspect → build → boot → observe → edit → rebuild → run

By the end of this lab, you should be able to build Luit, boot it under QEMU, use its shell, observe processes and memory, test filesystem persistence, locate major source subsystems, and add one small user program without changing the kernel.

1.1 Learning outcomes

After completing this lab, you should be able to:

1. distinguish the Linux host machine from the emulated RISC-V guest machine;
2. explain the roles of the cross-compiler, QEMU, OpenSBI, the Luit kernel, LuitFS, `init`, and the shell;
3. identify the important build artifacts: `kernel.elf`, user ELF, `fs.img`, and `mkfs/mkfs`;
4. use core shell composition mechanisms: pipes, redirection, sequencing, and background execution;
5. observe process states and physical-memory use;
6. distinguish persistence from crash consistency;
7. navigate the source tree using a subsystem-oriented map; and
8. complete a full edit–build–boot–run cycle.

1.2 What Lab 0 is not

- It is not a race to type commands. Read the output and connect it to the system structure.
- It is not a kernel implementation exercise. Do not modify files under `kernel/` or `hal/`.
- It is not a search for hidden tasks. The student bundle intentionally contains no later-lab specifications or solutions.
- It is not a Linux administration exercise. Do not install packages on shared laboratory machines.

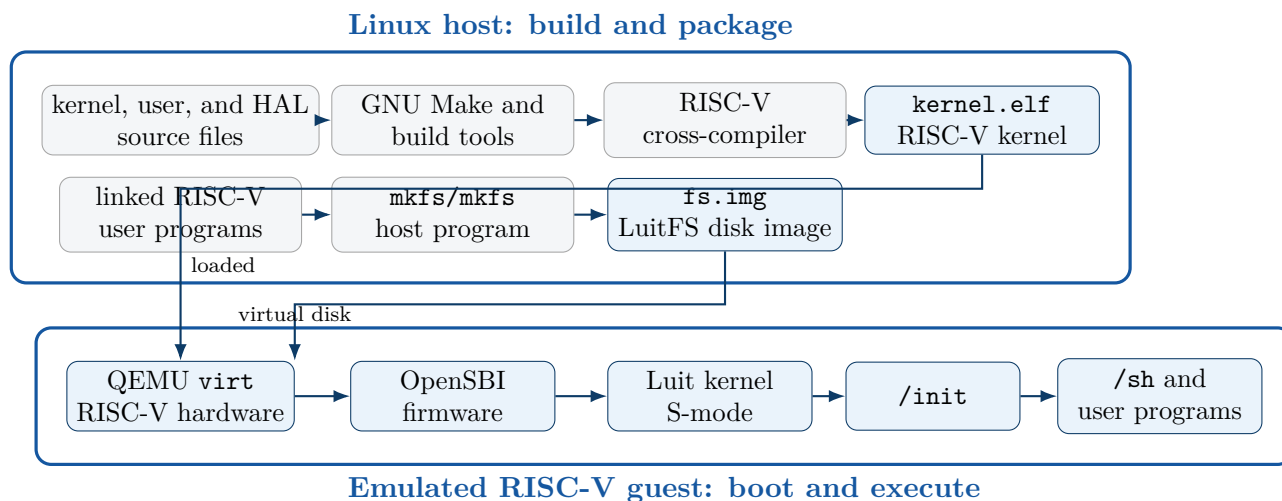
1.3 Suggested working plan

Phase	Activity	Time
A	Tool check and package inspection	15 min
B	Build, identify artifacts, and boot	30 min
C	Shell, files, pipes, and processes	40 min
D	Persistence experiment	20 min
E	Guided source-tree tour	35 min
F	Add and run <code>hello</code>	30 min
G	Complete report evidence	10 min

2 The System Before You Touch It

The first organisational idea to learn is that two different computers are involved, even though you see only one physical laboratory machine.

2.1 Two machines and four important boundaries



The host/guest rule

Commands such as `make`, `gcc`, `unzip`, and `mkfs/mkfs` run on the Linux **Host**. The file `kernel.elf` and the programs stored in `fs.img` execute inside the RISC-V **Guest**. A target executable cannot normally run directly on the x86-64 host.

Read the heading above every command block

This handout uses two different command environments:

- **LINUX HOST TERMINAL:** run the commands in the ordinary Linux terminal from the extracted source directory. These commands inspect source files, build Luit, and start QEMU.
- **LUIT GUEST SHELL:** run the commands only after Luit has booted and the `luit$` prompt is visible.

The prompt itself is not included in executable command blocks, so the commands remain copy-pasteable. If you currently see `luit$`, GNU commands such as `sed`, `find`, and option-rich `grep` commands will not work. Exit QEMU with **Ctrl-A**, release, then **x**, and return to the Linux host terminal.

2.2 The three artifacts you should always recognise

Artifact	Runs/used where?	Purpose
<code>kernel.elf</code>	RISC-V guest	The linked Luit kernel loaded by QEMU/OpenSBI.
<code>user/echo</code> , <code>user/sh</code> , etc.	RISC-V guest	Separately linked user-mode ELF programs.
<code>fs.img</code>	Attached to guest	A persistent LuitFS disk image containing user programs and files.
<code>mkfs/mkfs</code>	Linux host	A native host utility that constructs <code>fs.img</code> .

3 Phase A - Verify the Development Tools

The laboratory machines should already contain the required tools. Print their versions:

LINUX HOST TERMINAL - run outside QEMU

```
riscv64-unknown-elf-gcc --version
riscv64-unknown-elf-ld --version
qemu-system-riscv64 --version
gdb-multiarch --version
make --version
python3 --version
```

If gdb-multiarch is unavailable, try:

LINUX HOST TERMINAL - run outside QEMU

```
riscv64-unknown-elf-gdb --version
```

A tool is missing

Stop and inform a TA. Do not use `sudo`, do not change the laboratory machine globally, and do not modify kernel source files to work around an environment problem.

Why the compiler name is long

`riscv64-unknown-elf-gcc` is a cross-compiler. It runs on the host's x86-64 Linux system but emits code for a bare-metal 64-bit RISC-V target. QEMU provides the RISC-V machine on which that code can execute.

Checkpoint: Tools available

Record the version number of the RISC-V compiler and QEMU. You should be able to explain why ordinary host `gcc` is not used to build the Luit kernel.

4 Phase B - Unpack, Inspect, and Build

4.1 Unpack the student release

Use the archive supplied by the instructor. The filename may be renamed for your section; the following uses the standard name.

LINUX HOST TERMINAL - run outside QEMU

```
unzip LUIT_Student_Release.zip
cd LUIT_Student_Release
ls
```

You should see entries similar to:

EXPECTED OUTPUT / DISPLAY - do not type this block

```
Makefile  README.md  kernel/  hal/  user/  mkfs/  tools/  docs/
```

Preserve your own work carefully and do not publish the course source publicly.

4.2 Inspect before building

Run the following from the repository root:

LINUX HOST TERMINAL - run outside QEMU

```
find . -maxdepth 2 -type d | sort
grep -n -A3 '^UPROGS' Makefile
sed -n '1,120p' README.md
```

Do not try to understand every file yet. At this stage, answer only three questions:

1. Where is kernel code stored?
2. Where are user programs stored?
3. Which file describes how the system is built?

4.3 Perform a clean build

LINUX HOST TERMINAL - run outside QEMU

```
make clean
time make -j$(nproc)
```

The build uses `-Wall -Werror`; therefore, a compiler warning is treated as a build failure. A clean release should build with no warnings and no errors.

Now run the same build command again without changing any file:

LINUX HOST TERMINAL - run outside QEMU

```
time make -j$(nproc)
```

The second command should do little or no work because `make` compares file timestamps and rebuilds only targets whose inputs have changed.

4.4 Classify the generated artifacts

LINUX HOST TERMINAL - run outside QEMU

```
file kernel.elf
file user/echo
file mkfs/mkfs
ls -lh kernel.elf fs.img kernel.asm kernel.sym
```

Observe that `kernel.elf` and `user/echo` are RISC-V executables, whereas `mkfs/mkfs` is a host executable.

Report Q1 - Build and architecture evidence

Record the first and second build times. Explain why the second build is faster. Then classify `kernel.elf`, `user/echo`, `mkfs/mkfs`, and `fs.img` as host-side or guest-side artifacts, with one sentence describing each.

5 Phase C - Boot Luit

5.1 Start the virtual machine

LINUX HOST TERMINAL - run outside QEMU

```
make qemu CPUS=1
```

QEMU creates a virtual 64-bit RISC-V **virt** machine, attaches **fs.img** as a VirtIO block device, loads OpenSBI, and supplies **kernel.elf** as the supervisor payload.

Look for the following stages rather than memorising exact output:

1. OpenSBI firmware banner;
2. the BrahmaputraOS/Luit banner;
3. device-tree discoveries such as memory, UART, interrupt controller, and hart count;
4. physical-memory, process, virtual-memory, and device initialisation;
5. the first user process; and
6. the shell prompt:

EXPECTED OUTPUT / DISPLAY - do not type this block

```
luit$
```

How to leave QEMU

Press **Ctrl-A**, release the keys, and then press **x**. This is a QEMU monitor escape sequence, not a Luit keyboard shortcut.

5.2 Boot path: Linux and Luit

Stage	Typical Linux	Luit under QEMU
Firmware	BIOS/UEFI initialises the platform	OpenSBI supplies RISC-V machine-mode services
Kernel loading	A boot loader such as GRUB loads Linux	QEMU/OpenSBI loads kernel.elf
Kernel entry	Architecture startup and Linux initialisation	kernel/entry.S then kernel/main.c
First user process	init/systemd	/init
Interactive interface	login and bash/zsh	/sh , showing luit\$

Report Q2 - Explain the boot chain

Write the Luit boot chain in order, beginning with QEMU and ending at the shell. For each of OpenSBI, the Luit kernel, **/init**, and **/sh**, state one responsibility.

6 Phase D - Explore the Unix Interface

6.1 Create data before trying file-processing commands

The initial filesystem contains programs but not a text file named **README**. Create your own data first:

LUIT GUEST SHELL - run only after the luit\$ prompt appears

```
ls
echo Luit is running > note.txt
cat note.txt
```



```

wc note.txt
grep running note.txt
grep '^Luit' note.txt

```

Luit's `grep` supports a compact regular-expression language including `^`, `$`, `.`, and `*`. It does not provide the large option set of GNU `grep`.

Try directories and links:

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```

mkdir demo
echo file inside demo > demo/inside.txt
ls demo
cat demo/inside.txt
ln note.txt note-link
cat note-link
rm note-link

```

6.2 Compose programs

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```

ls | wc
cat note.txt | wc
grep Luit note.txt | wc
ls ; ps

```

The shell operators expose core OS abstractions:

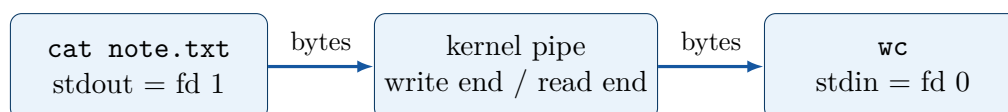
Syntax	Abstraction	Kernel work involved
	pipe	creates an in-kernel byte channel and connects file descriptors
>	output redirection	opens/creates a file and makes it descriptor 1
<	input redirection	opens a file and makes it descriptor 0
;	sequencing	waits for one command path and then starts the next
&	background execution	starts a command without waiting for it

Shell limitations are intentional

The current Luit shell does not implement quoting, environment variables, command substitution, wildcard expansion, or append redirection `»`. Use simple space-separated arguments and `>` only.

6.3 Understand one pipe

Conceptually, `cat note.txt | wc` connects the two command processes as follows:



The command programs are `cat` and `wc`. The shell implementation may also create a helper/coordinator process to construct and wait for the pipeline. Read the PIPE case in `user/sh.c` to distinguish command processes from shell-management processes.

Report Q3 - Unix composition

Draw the file-descriptor connection for `cat note.txt | wc`. Identify which descriptor of `cat` writes to the pipe and which descriptor of `wc` reads from it. After reading the PIPE case in `user/sh.c`, state how many new processes the current shell creates to execute the complete pipeline and what each process does.

7 Phase E - Observe Processes and Memory

7.1 Process states

First inspect the ordinary process table:

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```
ps
```

Now run a multi-process demonstration in the background and redirect its output so that the shell remains usable:

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```
vdemo 4 50 > vdemo.log &
ps
meminfo
ps
cat vdemo.log
```

Run `ps` promptly after the background command. Depending on timing, you may observe processes in states such as `run`, `runble`, `sleep`, or `zombie`. The exact snapshot is timing-dependent.

7.2 Virtual and physical memory

Run the demonstration in the foreground if you did not see its full output:

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```
vdemo 3 6
```

Each child prints the address of the same global variable. The virtual address is the same in each process, yet each process observes a private value. This is the address-space abstraction: identical virtual addresses can map to different physical pages.

`meminfo` reports free physical pages. Each page is 4 KiB. The value can change as processes allocate memory and return toward its earlier value when processes exit. Small temporary differences are normal because the observation itself also runs a process.

Report Q4 - Process and memory observations

Include one `ps` snapshot and one `meminfo` snapshot taken while `vdemo` is active. List the process states you observed. Explain why all `vdemo` children print the same virtual address but maintain different values.

8 Phase F - Test Persistence Carefully

8.1 Create a file and stop after the write is complete

At the shell prompt, replace the example roll number with your own:

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```
echo 210101099 > myroll.txt
cat myroll.txt
ls
```

Wait until the `luit$` prompt has returned. Then leave QEMU using **Ctrl-A**, release, and **x**. Do not run `make clean`.

Boot again:

LINUX HOST TERMINAL - run outside QEMU

```
make qemu
```

Then check:

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```
ls
cat myroll.txt
```

The completed file should remain because its blocks are stored in `fs.img`, which is a host file attached to the guest as a virtual disk.

Persistence is not the same as crash consistency

LuitFS v1 is persistent, but it does not yet provide transactional crash recovery. A completed, quiescent write should survive stopping and restarting QEMU. Power loss in the middle of a metadata update can still leave inconsistent state. This experiment tests persistence after a completed command; it does not prove crash consistency.

8.2 Understand what destroys the disk image

`make clean` deletes `fs.img`. The next build runs `mkfs/mkfs` and creates a fresh filesystem containing the configured user programs. Runtime-created files such as `myroll.txt` then disappear.

Changing the `UPROGS` list can also cause `fs.img` to be regenerated, because the disk image must be rebuilt to include the new program. This will matter in the final phase when you add `hello`.

Report Q5 - Persistence reasoning

State whether `myroll.txt` survived. Explain where the data was stored. Then explain why `make clean` followed by a rebuild removes it. Finally, explain why this successful experiment does not prove that LuitFS is crash-consistent.

9 Phase G - Learn the Source Organisation

Do not read the repository as one long collection of files. Read it in layers and follow one execution path at a time.

9.1 Top-level map

Path	Role	Questions it helps answer
Makefile	build graph and run commands	What is compiled? What enters <code>fs.img</code> ? How is QEMU launched?
kernel/	architecture-independent kernel core plus RISC-V entry/trap code	How are processes, memory, traps, files, and system calls implemented?
hal/	hardware abstraction interface and platform backends	Which code talks to UART, timer, interrupts, and block hardware?
user/	shell, utilities, user library, startup code, and tests	What runs outside the kernel? How do programs request kernel services?
mkfs/	host-side filesystem-image builder	How are user programs placed into a fresh LuitFS image?
tools/	build-time generators	Which source interfaces are generated from tables or scripts?
docs/	ABI, filesystem, debugging, and demonstration notes	What contracts and formats should remain stable?

9.2 Subsystem map inside the source

Files	Responsibility
kernel/entry.S, kernel/start.c, kernel/main.c	first instructions, privilege transition, device discovery, kernel initialisation, secondary harts
kernel/fdt.c, hal/hal.h, hal/qemu_virt/	device-tree parsing and platform-specific console, timer, interrupt-controller, and block-device operations
kernel/palloc.c, kernel/vm.c	physical-page allocation and page-table management
kernel/defs.h, kernel/proc.c, kernel/swtch.S	process data structures, process lifecycle, scheduling, and context switching
kernel/uservec.S, kernel/trap.c, kernel/kernelvec.S	transitions into and out of the kernel for system calls, faults, and interrupts
kernel/syscall.tbl, kernel/syscall.c, kernel/sysfile.c	system-call numbers, dispatch, and file-related system-call handlers
kernel/file.c, kernel/pipe.c, kernel/fs.c, kernel/bio.c	open-file objects, pipes, filesystem logic, and buffer cache
user/sh.c, user/ulib.c, gen- erated user/usys.S	shell parsing/execution, user library, and system-call stubs
mkfs/mkfs.c	creation of a new LuitFS image on the host

9.3 Use a three-pass reading method

1. **Pass 1 - map:** identify directories and filenames without reading function bodies.
2. **Pass 2 - control flow:** read one path top to bottom, such as boot in `kernel/main.c`.
3. **Pass 3 - interface crossing:** trace one user operation across the user/kernel boundary.

9.4 Guided tour 1: read the boot map

Return to the Linux host before inspecting source

All commands in Phase G operate on files in the extracted repository. If your prompt is `luit$`, you are still inside the guest. Exit QEMU with **Ctrl-A**, release, then **x**. Confirm that you are back in the directory containing `Makefile`, `kernel/`, and `user/` before continuing.

Run:

LINUX HOST TERMINAL - run outside QEMU

```
pwd
ls Makefile kernel user hal
sed -n '1,220p' kernel/main.c
sed -n '1,140p' hal/hal.h
ls hal/qemu_virt
```

In `kernel/main.c`, identify the order in which the following are initialised:

- device-tree information;
- console and interrupt controller;
- physical memory and processes;
- kernel virtual memory;
- buffer cache, inode cache, and file table;
- block device and traps; and
- the first user process.

Do not memorise every function. Learn the dependency idea: later subsystems rely on earlier ones.

9.5 Guided tour 2: locate a process

LINUX HOST TERMINAL - run outside QEMU

```
grep -n -A24 '^struct proc {' kernel/defs.h
grep -n '^struct proc *proc\[NPROC\];' kernel/proc.c
grep -n '^void scheduler' kernel/proc.c
```

The type is defined in `kernel/defs.h`. The process table and process-management code are in `kernel/proc.c`. This separation between data-structure declarations and implementation is common in C systems code.

9.6 Guided tour 3: trace one output operation

Use the source to follow this conceptual path:

```
user/echo.c → user/ulib.c → user/usys.S → kernel/trap.c → kernel/syscall.c →
kernel/sysfile.c → kernel/file.c → kernel/console.c → hal/qemu_virt/uart.c
```

The exact path depends on the destination file descriptor, but the important organisation is stable:

1. a user program calls a user-library function;
2. a generated stub executes the RISC-V system-call instruction;
3. trap code enters the kernel;
4. the dispatcher selects a handler;
5. the kernel operates on a file object; and
6. a device-specific HAL implementation eventually moves bytes to the UART when the destination is the console.

Report Q6 - Source organisation

Create a table with at least six rows. Each row must name one subsystem, its main source file(s), whether it is primarily host-side or guest-side, and one responsibility. Include boot, processes, memory, system calls, filesystem, and hardware abstraction.

Report Q7 - Process structure

From `kernel/defs.h`, choose four fields of `struct proc`. For each field, state its C type and explain its purpose in your own words. Do not copy nearby comments without explanation.

10 Phase H - Complete One Edit–Build–Boot–Run Cycle

You will add a user program. You will not change the kernel.

10.1 Create `user/hello.c`

Create and edit this file in the **Linux host terminal**, not inside the Luit shell. Replace the placeholders with your own details:

FILE: `user/hello.c` - create on the Linux host

```
#include "ulib.h"

int
main(int argc, char *argv[])
{
    printf("hello from YOUR_NAME, roll YOUR_ROLL_NUMBER\n");
    exit(0);
}
```

The arguments are intentionally unused in this simple program. The current build flags do not enable the additional warning that would reject unused parameters.

10.2 Add the program to the disk-image build

Open `Makefile`, find `UPROGS`, and add `user/hello` using the same spacing and continuation style as the existing entries. Do not add a filename extension in `UPROGS`.

For example, the relevant idea is:

MAKEFILE FRAGMENT - edit this on the Linux host

```
UPROGS = ... user/hello ...
```

Do not copy that abbreviated line literally; edit the actual list without deleting existing programs.

10.3 Build and observe what changed

LINUX HOST TERMINAL - run outside QEMU

```
make -j$(nproc)
```

Observe which files are rebuilt. At minimum, `user/hello` must be linked and `fs.img` must be regenerated so that the new executable appears on the guest disk.

Because `fs.img` is regenerated, files created during the earlier persistence exercise may disappear. That is expected and demonstrates the difference between changing source/build inputs and merely rebooting an unchanged disk image.

10.4 Boot and run

LINUX HOST TERMINAL - run outside QEMU

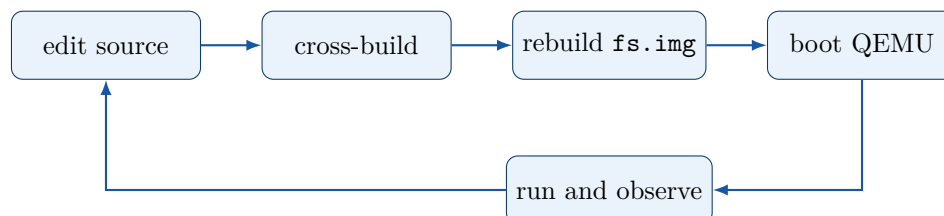
```
make qemu
```

At the Luit prompt:

LUIT GUEST SHELL - run only after the `luit$` prompt appears

```
ls  
hello
```

You have now completed the development loop used throughout the course:

**Report Q8 - Your first program**

Include your complete `user/hello.c`, the exact `UPROGS` fragment after your edit, and the terminal output from running `hello`. State why adding a user program caused `fs.img` to be rebuilt.

11 Host and Guest Quick Reference

What you are doing	Where to do it	Examples
Inspecting or editing source	Linux host terminal	<code>grep</code> , <code>sed</code> , editor, <code>find</code>
Building the OS	Linux host terminal	<code>make</code> , <code>make clean</code>
Starting or stopping QEMU	Linux host terminal / QEMU escape	<code>make qemu</code> ; Ctrl-A then x
Running Luit programs	Luit guest shell	<code>ls</code> , <code>cat</code> , <code>ps</code> , <code>meminfo</code> , <code>hello</code>
Inspecting kernel source	Linux host terminal only	<code>sed -n '1,220p' kernel/main.c</code>

One-line test

If the visible prompt is `luit$`, you are inside the guest and cannot inspect `kernel/main.c`. If the current directory contains `Makefile`, `kernel/`, and `user/`, you are on the Linux host in the source repository.

12 What to Submit

Keep the reports of this week's lab stored your PC. We will check the reports randomly at any time.

13 Getting Unstuck

Problem	Action
QEMU will not quit	Press Ctrl-A, release, then x.
The shell appears confused after a background command	Stop QEMU and reboot. Avoid backgrounding a command that reads from the console.
<code>myroll.txt</code> vanished after adding <code>hello</code>	Adding <code>hello</code> rebuilt <code>fs.img</code> ; this creates a fresh disk image.
A file vanished after <code>make clean</code>	<code>make clean</code> deletes <code>fs.img</code> . Rebuilding creates a new filesystem.
<code>></code> gives unexpected behaviour	Append redirection is not implemented in the current shell. Use <code>></code> only.
You cannot find <code>struct proc</code> in <code>proc.c</code>	The type is declared in <code>kernel/defs.h</code> ; <code>proc.c</code> contains the table and implementation.

14 The Main Ideas to Carry Forward

Before leaving the laboratory, make sure these statements are meaningful to you:

Idea	What you should now understand
Build boundary	The Linux host uses a cross-toolchain to create RISC-V kernel and user executables.
Boot boundary	QEMU emulates hardware, OpenSBI supplies firmware services, and Luit runs as the supervisor-mode kernel.
User/kernel boundary	User programs are separate ELF's; the shell composes them using system calls, file descriptors, processes, and pipes.
Memory abstraction	Processes share physical hardware while receiving private virtual address spaces.
Storage abstraction	A persistent disk image can survive reboot even when the filesystem is not fully crash-consistent.
Source organisation	Boot, HAL, memory, processes, traps, system calls, files, and user space form distinct but connected subsystems.
Working method	Edit a small part, build cleanly, boot, run a targeted test, and explain the evidence.

Welcome to Luit. Read it, run it, measure it, and then learn to change it.